Core idea and data model

Live: https://miraflores-workpaper-okafor.netlify.app · **Code:** see the repository link in the submission

Core idea: every number is a claim with its own case file

A workpaper exists to prove the return, so the unit of the design is not a cell or a schedule. It is a **claim**: "line 26 is 12,000". Click any number anywhere and the **Inspector** opens its case file:

- the engine value next to the workpaper value
- how it was built, and the exact field on the source document, highlighted
- how it was verified and by whom
- every change, with who, why and tax effect
- the notes, questions and checks on it

Everything else is a view over these claims: the issues inbox, the area workspaces, the document index ("where it landed, or why not"), the season and filing timeline, the client portal and the examination record. Nothing is overwritten, and nothing is a spreadsheet.

The engine is tested, not trusted. About 25 independent checks recompute figures from the printed documents, IRS/FTB records, client answers and the prior year, then compare them with the engine. Each finding comes with evidence and a priced fix that can be undone. Checks that pass are kept as evidence too.

Data model: immutable packet + append-only log → projection

```
client packet (ingested once, never changed)  +  events (only appended; undo is an event too)
   documents · values read · engine lines        verify · adjust · note · ask · judge · sign off
   + trace graph · prior year · agency records   file · e-file result · view sensitive · log time
                          └──────────►  project(packet, events, as-of date)  ◄──────────┘
            pure function: the server runs it to enforce rules, the browser to render
```

- **Ingestion.** The packet is validated against a versioned schema and normalized into Postgres in one transaction. It is idempotent by content hash and serialized per return. Lot detail is paged from the server.
- **One log gives:** undo, history, "changed since I last looked", freeze at filing with an amendment layered on top, and view logs (an SSN reveal is logged before the value is returned).
- **Sign-offs reopen themselves.** Each one stores a fingerprint of its area, and any later change reopens it.
- **Re-anchoring.** Every action stores a meaning-based anchor (document + field) and the value it was based on. Work follows a corrected document or a full pipeline re-run (all 3,012 ids renumbered). It goes stale when the value changes, and is flagged if the value disappears.
- **Tax effect without re-running the engine:** the change × the version's per-dollar rate, flowed up through linear sums. Payments and credits move 1:1.
- **One rule book.** `can(actor, event, state)` encodes the firm's standards: no self-review, partner approval over $1,000, an authority on every judgment, open review notes block filing, masking without clearance. The server enforces it; the UI uses it to explain disabled buttons.
- **Snapshots.** Writes read only new log rows. Each return's headline state is stored, so firm status never replays a log.

Tradeoffs

Built now	At scale
Polling every 3 s; a stale write to the same item is rejected (409)	Server push
Masking + logged unmask on the server	Field encryption; per-return keys destroyed after 7 years
Role switcher, no passwords (rules still enforced on the server)	Real auth and row-level security by firm
Tax effects estimated from per-dollar rates	Re-run the engine on the workpaper layer
Simulated documents behind a renderer interface	PDF renderer using the same document/page/field links

How AI was used: I built this with Claude Code as the engineer. I set the requirements and chose the architecture's scaling priorities, including re-anchoring work across pipeline re-runs. I verified every error it found against the printed document text. Stack: Next.js on Netlify, Postgres on Neon; 18 tests, including a full-season test.